



Synthetic Price Path Generation for European Gas Markets Using Generative Adversarial Networks with Applications to Option Pricing

Yeva Galstyan

Master's Thesis, Department of Applied Computer Science,
Fulda University of Applied Sciences

Matriculation number: 1507481

Supervisor: Prof. Dr. Christoph Scheich

Co-supervisor: Dr. Alexander Jungwirth

Submitted on September 15, 2026

Abstract

Here goes abstract.

Contents

Contents	II
List of Figures	IV
List of Tables	IV
1 Introduction	1
1.1 Context	1
1.2 Problem Statement and Motivation	1
1.3 Goals	1
1.4 Own Contribution	1
2 Fundamentals	2
2.1 European Natural Gas Markets	2
2.2 Futures Contracts	3
2.3 Stylized Facts of Commodity Returns	3
2.4 Options and Implied Volatility	3
2.5 Generative Adversarial Networks	4
2.5.1 Basic Generative Adversarial Networks	4
2.5.2 cGANs	6
2.5.3 ForGAN	6
2.5.4 Fin-GAN	7
2.6 Related Work	9
3 Methodology and Implementation	10
3.1 Data Pipeline	10
3.1.1 Source Data	10
3.1.2 Front-Month Series Construction	10
3.1.3 Return Definition	11
3.1.4 Data Split	12
3.2 Model Architecture and Adaptation	12
3.2.1 Architecture	12
3.2.2 Training configuration	13
3.2.3 Adaptations	13
3.3 Model Configurations and Ablations	13
3.4 Benchmark Models	13
3.5 Evaluation Methodology	13
3.6 Option Pricing Setup	13
3.7 Software Environment	14
4 Experiments	15

5	Discussion	16
6	Conclusion and Outlook	17

List of Figures

1	Structure of a standard GAN	5
2	Structure of a cGAN	6
3	TTF train, validation and test data split	12

List of Tables

1	Software packages and versions	14
---	--	----

1 Introduction

1.1 Context

1.2 Problem Statement and Motivation

1.3 Goals

1.4 Own Contribution

2 Fundamentals

2.1 European Natural Gas Markets

Natural gas is a physical commodity that exists within a gas grid. The Title Transfer Facility (TTF) is a virtual trading hub in the Netherlands. TTF is operated by Gasunie Transport Services (GTS) within the Dutch gas transmission network. A virtual trading hub allows participants to transfer ownership without reference to a specific physical location. Gas futures contracts referencing TTF are traded on ICE Endex, the European energy exchange operated by Intercontinental Exchange.

A market is considered liquid if many participants can buy or sell continuously, without affecting the asset price. TTF is the most liquid natural gas market in Europe, with traded volumes 4.5 times the combined volume of the other eight main European hubs (Heather 2026). Therefore, the TTF gas price serves as the reference price for gas contracts.

At virtual hubs such as TTF, gas price formation is determined by short-run supply and demand (Obadi and Korcek 2020). On the demand side, natural gas consumption is seasonal, although deviations of weather from the seasonal norm also affect prices (Brown and Yücel 2008). Industrial production is the second source of demand. In the chemical sector, for example, gas is used as an energy source and raw material. Gas also competes with coal in power generation. As a result, the gas market is linked to the coal market and, through emission allowances, to the carbon market. On the supply side, natural gas prices depend on pipeline flows and deliveries of liquefied natural gas (LNG), which is natural gas cooled to liquid form for transport by ship.

As has been observed repeatedly in the past, the European natural gas market is highly sensitive to supply disruptions. The European gas price crisis between 2021 and 2022 is the most recent episode. During the COVID-19 pandemic in 2020, energy demand collapsed. This was followed by economies reopening and gas demand recovered faster than production could follow. From mid-2021, post-pandemic increase in demand was also combined with reduced Russian pipeline flows, strong Asian competition for LNG cargoes and storage levels below the seasonal norm (Heather 2022). As a result, TTF gas prices rose from roughly €18/MWh in the first quarter of 2021 to a peak of €339/MWh on 26 August 2022 and then declining from early 2023 and onward.

The European gas price crisis eventually raised the question of whether the TTF price benchmark had become less representative of the underlying gas market. In November 2022, the European Commission proposed a Market Correction Mechanism (MCM), which would cap gas prices during crisis periods (European Commission 2022). The mechanism applied until January 2025 but was never activated, as gas prices declined after 2023.

2.2 Futures Contracts

Futures contracts are standardized agreements to buy or sell an asset at a fixed price on a specified future date. The quantity and the delivery date are set by the exchange. The price is agreed between the parties and remains fixed for the contract. As a result, all contracts for a given month are identical in their terms and can be freely transferred. TTF gas futures are quoted in EUR/MWh and are physically delivered, with delivery spread evenly across the hours of the delivery month.

Price formation of natural gas in Europe has generally taken place on the futures rather than spot market (Obadi and Korcek 2020). A futures contract can be traded repeatedly before maturity. As a result, the market is open for participants who hedge or speculate without an interest in the physical commodity.

At any point in time, contracts for multiple consecutive months are listed and can be traded in parallel. The contract with the nearest delivery on a given day is the front-month contract (M1), and the next is the second-month contract (M2). Trading concentrates in the front-month contract, hence it is commonly used as the reference for the current gas price.

2.3 Stylized Facts of Commodity Returns

2.4 Options and Implied Volatility

2.5 Generative Adversarial Networks

2.5.1 Basic Generative Adversarial Networks

Generative modeling is an unsupervised learning approach. A generative model takes a set of training examples and learns the probability distribution that produced them. Generative Adversarial Networks (GANs) are generative models first introduced by I. J. Goodfellow et al. (2014). GANs have been particularly successful in image and video generation. They have also been used to create synthetic data, for training other machine learning models and to fill in gaps in missing data (I. Goodfellow et al. 2020).

GANs are based on game theory, which studies situations where several decision-makers are present. Each participant chooses an action, and the outcome depends on the choices of all of the participants. A GAN has two decision-makers, **the generator (G)** and **the discriminator (D)**, typically implemented as neural networks. G takes a vector of random noise as input and generates a sample as output. D takes a real or generated data sample as input and outputs a probability that the input sample is real. The goal of G is to fool D by generating realistic-looking data samples. The goal of D is to distinguish real samples from generated ones.

Assume that the data x follows an unknown distribution p_{data} , from which new samples need to be generated. The generator is a neural network $G(z; \theta_g)$, where θ_g denotes the learnable parameters. The input z is a noise vector drawn from a prior distribution p_z . Gaussian distribution is normally used to define p_z ; the GAN framework does not add limitations. The probability distribution of the generated samples is denoted p_g . The discriminator is a neural network $D(x; \theta_d)$, with learnable parameters θ_d . It outputs the estimated probability that the input was drawn from p_{data} rather than p_g . G and D have cost functions, respectively $J^{(G)}(\theta_d, \theta_g)$ and $J^{(D)}(\theta_d, \theta_g)$. Both G and D need to minimize their cost functions, while they can only control their learnable parameters, respectively θ_g and θ_d . Hence, the solution is a Nash equilibrium, rather than a minimum of a single cost function.

The GAN framework is flexible in the sense of the cost functions applied to G and D . The simplest version of the game is a zero-sum game. In this, the sum of all of the cost functions is always zero.

$$J^{(G)}(\theta_d, \theta_g) = -J^{(D)}(\theta_d, \theta_g). \quad (1)$$

The cost function of D is defined as a standard cross-entropy

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{\mathbf{x} \sim p_{data}} \log D(\mathbf{x}) - \frac{1}{2} \mathbb{E}_{\mathbf{z}} \log(1 - D(G(\mathbf{z}))). \quad (2)$$

Learning the zero-sum game minimizes the Jensen-Shannon divergence between p_{data} and p_g . Convergence holds only if both players are updated in function space. In practice, D and G are neural networks and updated in parameter space. D minimizes a cross-entropy while G maximizes the same quantity. Generated samples are rejected with high confidence at the start of training, and the gradient of G vanishes at the point at which it performs worst (I. Goodfellow 2017).

Therefore the cost function of G is constructed by flipping the target of the cross-entropy.

$$J^{(G)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{p_z} [\log[D(G(z; \theta_g); \theta_d)]] . \quad (3)$$

The cost function of D and G do not sum to zero anymore, hence the game is no longer zero-sum. Alternative cost functions have been introduced over time. One example is Wasserstein GAN, which replaces the Jensen–Shannon divergence between p_{data} and p_g with the Wasserstein-1 distance (Arjovsky et al. 2017).

Figure 1 is the visualization of a standard GAN workflow adapted from (Vuletić et al. 2024).

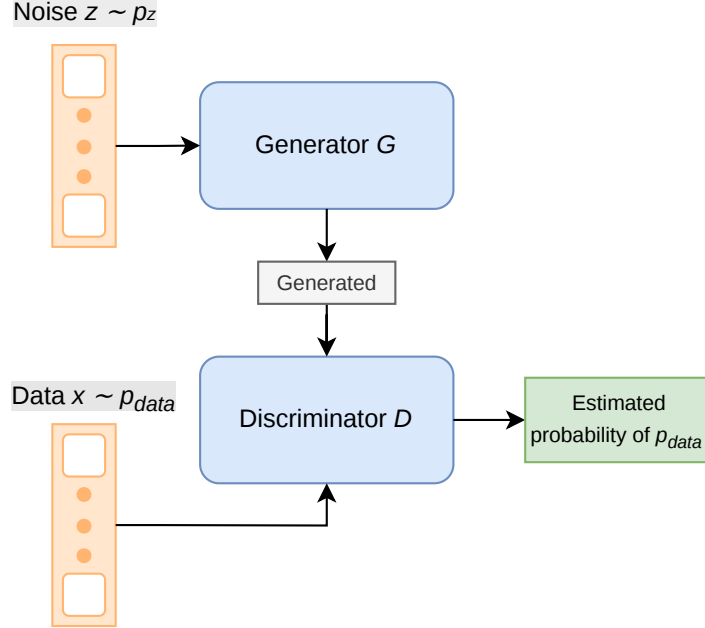


Figure 1: Structure of a standard GAN

GANs are trained by updating the two networks in turn. At each step, a minibatch of samples and noise vectors are drawn from p_{data} and p_z respectively. D is updated with respect to θ_d while θ_g is fixed. G is updated with respect to θ_g while θ_d is fixed. D is commonly trained k times for every update of G , although I. Goodfellow (2017) argues that $k = 1$ is sufficient. k is defined as an independent training parameter.

As stated previously, convergence is not guaranteed in practice. This means that the training of a GAN can settle in a state where $p_g \neq p_{\text{data}}$. The most common such state is mode collapse. It occurs when G repeatedly maps noise vector to very similar outputs. As a result, p_g covers only a small part of p_{data} but D assigns a high probability

that the generated samples are real. Mode collapse is still an unresolved challenge in GAN training. In recent years, several strategies have been proposed to mitigate its effect, such as stronger generators and discriminators, longer training and regularization. However, each of these approaches comes with drawbacks (Barsha and Eberle 2025).

2.5.2 cGANs

Standard GANs can be extended to conditional models (I. J. Goodfellow et al. 2014). This extension was further developed by Mirza and Osindero (2014). In Conditional Generative Adversarial Networks (cGANs), the data follow a conditional distribution $p_{\text{data}}(x | c)$, where c is auxiliary deterministic information. The condition is applied to D and G as an additional input. The cost functions for G and D are the same as for a standard GAN with the condition c .

$$J^{(D)}(\theta_d, \theta_g) = -\frac{1}{2} \mathbb{E}_{x \sim p_{\text{data}}} \log D(x | c) - \frac{1}{2} \mathbb{E}_z \log(1 - D(G(z | c) | c)). \quad (4)$$

Figure 2 is the visualization of a cGAN workflow adapted from (Vuletić et al. 2024).

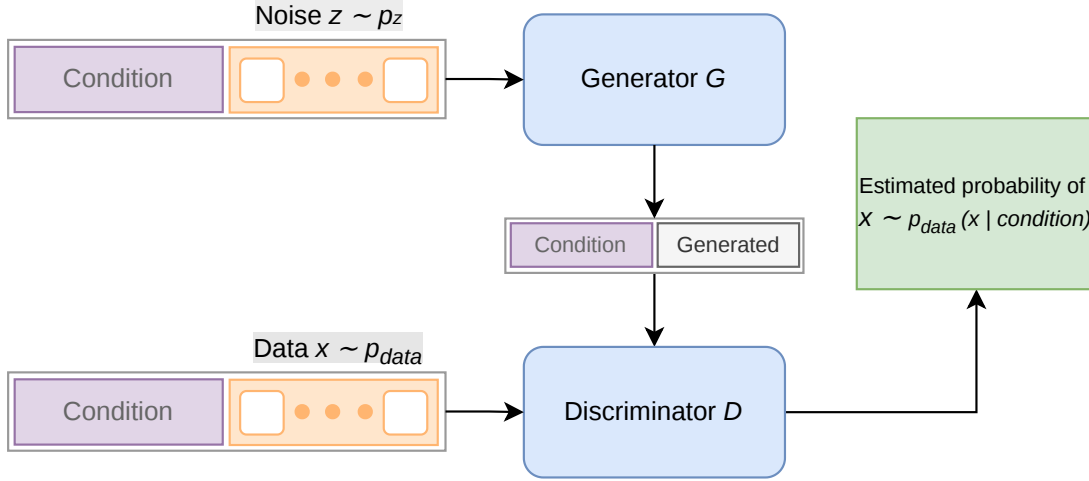


Figure 2: Structure of a cGAN

2.5.3 ForGAN

The conditional input proposed in cGANs by Mirza and Osindero (2014) prepared the ground for domain-specific GAN architectures. ForGAN was introduced specifically for time series forecasting (Koochali et al. 2019).

ForGAN is trained to approximate the conditional distribution of the next value in a time series. The condition is the sequence of the previous l observations, ending at time t

$$c = (x_{t-(l-1)}, \dots, x_t). \quad (5)$$

A sliding window of $l + 1$ observations is moved along the series. At each position, the first l observations form c and the last is x_{t+1} . G takes a noise vector z and c and outputs a value for the target x_{t+1} . D takes a value of x_{t+1} and c and outputs the probability that x_{t+1} is from $p(x_{t+1} | c)$.

A full probabilistic distribution of x_{t+1} can be obtained through repeated sampling of G with different noise vectors and the same c .

ForGAN adds a recurrent layer to G and D . The layer processes c as an ordered sequence and constructs a vector representation. The recurrent layer can be a long short-term memory (LSTM)(Hochreiter and Schmidhuber 1997) or a gated recurrent unit (GRU) (Chung et al. 2014).

2.5.4 Fin-GAN

Fin-GAN was introduced as an extension of ForGAN (Vuletić et al. 2024). It focuses on the probabilistic forecasting of financial time series, specifically equity returns. The ForGAN architecture and the cost function of D are unchanged. LSTM is chosen as the recurrent layer.

The motivation behind Fin-GAN is to correctly forecast the sign of the return, since the sign determines whether the trade is profitable. For this purpose, economics-driven terms are introduced into the cost function of G . In contrast to the unsupervised setting of standard GANs, Vuletić et al. (2024) classify Fin-GAN as supervised learning.

The terms are defined over a minibatch of n_{batch} pairs. For each condition in the batch, G produces a forecast $\hat{x}_i$ of the realized value x_i . Let $\hat{x} = (\hat{x}_1, \dots, \hat{x}_{n_{\text{batch}}})$ and $x = (x_1, \dots, x_{n_{\text{batch}}})$.

Fin-GAN introduces four economics-driven terms to the cost function of G . $J^{(G)}$ can be the zero-sum, cross-entropy or Wasserstein cost function. Vuletić et al. (2024) report that the Wasserstein cost function was numerically unstable during training. Hence $J^{(G)}$ is the cross-entropy cost function given in Eq. 3. The four remaining terms are defined below.

$$L^G(x, \hat{x}) = J^{(G)}(x, \hat{x}) - \alpha \text{PnL}^*(x, \hat{x}) + \beta \text{MSE}(x, \hat{x}) - \gamma \text{SR}^*(x, \hat{x}) + \delta \text{STD}(x, \hat{x}). \quad (6)$$

A positive forecast corresponds to buying an asset, and a negative forecast to selling it short. The profit and loss PnL of the trade is the sign of the forecast multiplied by the realized return $\text{PnL}(x_i, \hat{x}_i) = \text{sgn}(\hat{x}_i) \cdot x_i$. Since only the sign enters the formula,

2 Fundamentals

the outcome does not depend on the magnitude of the forecast. $\text{sgn}(\hat{x}_i)$ is not differentiable, so the standard term PnL would not provide a gradient for training. A smooth approximation is introduced

$$\text{PnL}_a^*(x_i, \hat{x}_i) = \tanh(k_{\tanh} \hat{x}_i) x_i, \quad (7)$$

where $k_{\tanh}$ is a hyperparameter that controls the accuracy of the approximation. Vuletić et al. (2024) use $k_{\tanh} = 100$, that is large enough to give a close approximation and small enough to maintain strong gradients.

Although the sign of the forecast is the main interest, forecasting an $\hat{x}_i$ that is close to x_i is also of importance. The mean squared error MSE measures the squared difference between the forecast and the realized value, averaged over the minibatch.

The Sharpe ratio SR measures risk-adjusted return and is defined as the mean of the PnL_a^* series divided by its standard deviation.

$$\text{SR}^*(x, \hat{x}) = \frac{\bar{p}}{\sqrt{\frac{1}{n_{\text{batch}}} \sum_{i=1}^{n_{\text{batch}}} (p_i - \bar{p})^2}}, \quad (8)$$

where $p_i = \text{PnL}_a^*(x_i, \hat{x}_i)$ and $\bar{p}$ is its mean over the n_{batch} forecasts. The standard deviation STD term is the denominator of Eq. 8. Logically maximizing SR^* is the same as maximizing PnL_a^* and minimizing STD at the same time. As a result, including all three terms in L^G is redundant. However, during training SR^* and the combination of PnL_a^* and STD produce different gradients (Vuletić et al. 2024).

All of the terms above are measured in different units, hence they produce incomparable gradients. As a solution, four additional hyperparameters $\alpha, \beta, \gamma, \delta$ are introduced. Fin-GAN is first trained on $J^{(G)}$ only, and gradient norms of other terms are calculated. Each hyperparameter is the mean of the ratios between the cross-entropy gradient norms and the gradient norms of the corresponding term. Large values of $\alpha, \beta, \gamma, \delta$ make the feedback of D insignificant and move the model towards supervised learning, but small values leave it in the standard GAN setting.

If the term is not included in the objective, the hyperparameter is set to zero. Several versions of L^G can be constructed from different combinations of terms. Only combinations that include economics-driven terms are considered for Fin-GAN. Vuletić et al. (2024) train eight such versions and select the combination with the highest Sharpe ratio on the validation set. Then, the selected G is evaluated on the test set.

2.6 Related Work

3 Methodology and Implementation

3.1 Data Pipeline

3.1.1 Source Data

The futures data was obtained from BASF's Bloomberg infrastructure and covers TTF natural gas futures for the first three nearby contracts between January 2010 and April 2026. The data is distributed as three files: contract prices, contract metadata, and monthly listings of active contracts.

Daily settlement prices. This file provides the price history of the contracts in the dataset. Each contract is observed once per trading day, identified by contract symbol and trading date. Each record contains opening, high, low and daily closing prices in EUR/MWh, trading volume and open interest. Coverage of these fields varies considerably with how actively a contract was traded and how far from expiry it is. The closing price is the exception and is available throughout the whole dataset. Additionally, it falls outside the intraday high–low range for 7,712 of 25,884 records, or 29.8%. The deviation is small in most cases, with a median of 0.10 EUR/MWh, but reaches up to 42.26 EUR/MWh. The latter confirms that it is a settlement price set by the exchange rather than a price at which trading occurred.

Contract metadata. This file provides static attributes per unique contract in the dataset. The expiry date provided per record is used to rank contracts by time to maturity in Sec. 3.1.2. All contracts in the dataset are physically delivered instruments traded on ICE Endex and quoted in EUR/MWh. First and last trading dates, notice dates and delivery dates are also recorded.

Contract listings. This file records which contracts were active on the market, as monthly snapshots taken on the first calendar day of each month. Each contract is observed once per snapshot. Together with the expiry dates, the listings determine which contract is the front month on a given date, as described in Sec. 3.1.2.

3.1.2 Front-Month Series Construction

As described in Sec. 3.1.1, the contract listings and the contract metadata each hold part of the information required to identify the front-month contract. The listings record which contracts were active in a given month, while the metadata records when each contract expires. Therefore, the expiry dates were joined onto the listings by contract symbol. As a result, each listing record carries the expiry date of the contract it refers to.

The contracts were then sorted by expiry date within each monthly snapshot and the earliest-expiring one was kept.¹ This gives the front-month contract for that month. Since the snapshots are monthly but the series must be daily, each trading day was assigned the front-month contract from the most recent snapshot on or before that

¹Trading in a TTF futures contract ceases two UK business days before the first day of the delivery month.

day. The settlement price of the selected contract was then added for each trading day, matching on trading date and contract symbol. The trading days without an available settlement price were discarded. The resulting series contains 3,998 trading days spanning 2010-01-04 to 2026-04-24, with settlement prices ranging from 3.509 to 339.196 EUR/MWh.

Since the front-month contract is replaced at each expiry, the series is not the price history of a single instrument. Returns spanning a contract transition reflect the change in the price level as well as the difference between the two contracts. Across the 195 transition days in the sample, the standard deviation of daily log returns is 0.071, compared to 0.037 on all other days. When the transition days are excluded, excess kurtosis falls from 14.93 to 14.67, and skewness from 0.71 to 0.26. Transition days contribute little to the tail weight of the series, but they are responsible for most of its positive skew.

The transition effect described above is preserved and quantified rather than removed, since restricting the analysis to a single contract would reduce the sample to a few months.

TODO: these needs to go somewhere else: As shown in Sec. 3.1.2, transition days contribute little to the tail weight of the series but account for a substantial part of its positive skew.

3.1.3 Return Definition

The original FinGAN input series is built from two returns per day (Vuletić et al. 2024). The first is the intraday return, from the opening to the closing price and the second is the overnight return, from one closing price to the next opening price. The return of the corresponding sector ETF is then subtracted from each entry. This isolates the part of the movement specific to the asset.

Neither step has been adopted for the TTF futures dataset. The two-return scheme requires opening and closing prices that are both traded prices. As shown in Sec. 3.1.1, the daily price in this dataset is a settlement price set by the exchange. A return computed between the opening trade and the settlement price would therefore combine two different price concepts.

Additionally, the excess-return step requires a benchmark that leaves a meaningful residual when subtracted. In the case of equities, subtracting the sector ETF return removes the market-wide component and leaves the movement specific to the individual stock. European natural gas has no comparable instrument.

As a substitute, the second nearby contract was considered. However, the difference between the first and second nearby returns is a calendar spread return. A calendar spread describes the term structure of the market rather than the level of prices. A model that is trained on it would learn how the gap between two contracts moves, not how the gas price itself moves. Option payoffs depend on the price level at expiry, so a model trained on spread returns could not be used to price options.

Therefore, the dataset was constructed from daily logarithmic returns of the front-month settlement price. Returns are approximately stationary and therefore comparable

3 Methodology and Implementation

across the sample period. Additionally, log returns are additive across time and allow price levels to be recovered exactly from simulated returns. The latter is required for the Monte Carlo procedure in Sec. 3.6.

$$x_t = \log\left(\frac{P_t}{P_{t-1}}\right), \quad (9)$$

where P_t is the front-month settlement price on trading day t . The transformation gives 3.997 return observations.

3.1.4 Data Split

The return series was split chronologically into training (80%), validation (10%), and test (10%) sets using the proportions of (Vuletić et al. 2024). Training covers 2010-01-05 to 2023-01-18 (3,197 returns), validation 2023-01-19 to 2024-09-04 (399 returns), and test 2024-09-05 to 2026-04-24 (401 returns). The chronological split was explicitly used to avoid look-ahead bias. Following the current split, the 2021 to 2022 price disruption discussed in Sec. 2.1 falls entirely within the training period, while the validation and test periods correspond to a calmer period, as seen in Figure 3.

Sliding window matrices were constructed for each split separately, so that no window spans a split boundary. The window was moved forward by one trading day.

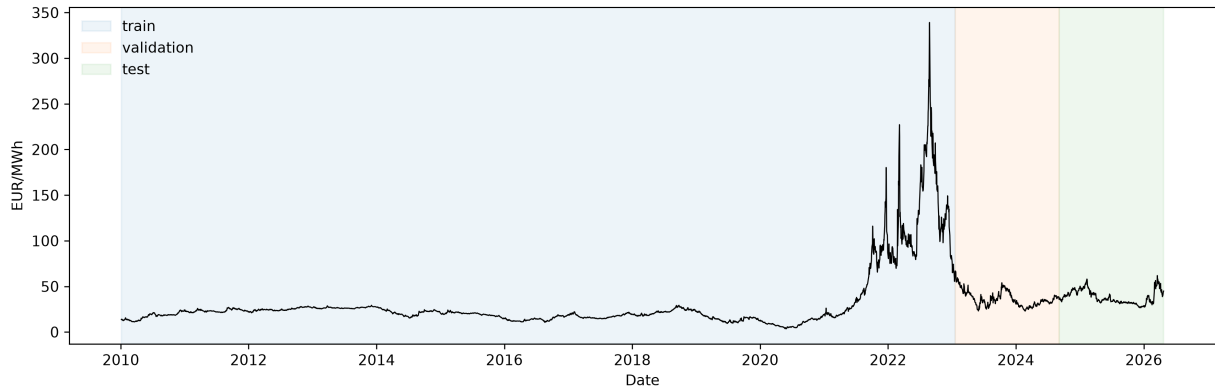


Figure 3: TTF train, validation and test data split

3.2 Model Architecture and Adaptation

3.2.1 Architecture

The ForGAN architecture described in Sec. 2.5.3 was used, without modifications. In G , the condition passes through an LSTM layer. The final hidden state is concatenated with the noise vector z . The result passes through a dense layer of width $R_G + N$, then

3 Methodology and Implementation

a ReLU activation and another dense layer of width one, where R_G is the LSTM hidden size and N is noise dimension.

In D , the candidate value ($\hat{x}_{t+1}$ or x_{t+1}) is appended to the condition, that gives a sequence of size $l + 1$. The result passes through an LSTM layer with the hidden size R_D . The final hidden state passes through a dense layer with a sigmoid activation.

Following the the implementation of Vuletić et al. (2024) $N = 8$, $R_G = 8$ and $R_D=8$. However $N = 8$, $R_D = 8$ are varied in Sec. 3.3.

3.2.2 Training configuration

3.2.3 Adaptations

TODO: The implementation is based on the reference Fin-GAN code published by Vuletić et al. (2024). The network definitions, the gradient norm matching procedure and the training loops were adopted from that implementation.

3.3 Model Configurations and Ablations

3.4 Benchmark Models

3.5 Evaluation Methodology

3.6 Option Pricing Setup

3.7 Software Environment

Experiments were run on a MacBook Pro with an Apple M4 Pro processor and 24 GB of RAM, running macOS 26.6.1. The NVIDIA CUDA toolkit is not supported on macOS. Training was on CPU. The software packages and their respective versions used for the data pipeline, model training and evaluation are listed in Table 1.

Package	Version
Python	3.12.13
PyTorch	2.13.0
NumPy	2.5.2
pandas	3.0.5
polars	1.43.2
pyarrow	25.0.1
DuckDB	1.5.5
SciPy	1.18.0
Matplotlib	3.11.1

Table 1: Software packages and versions

The TTF futures data from Bloomberg and the options data from Databento are licensed and cannot be distributed. The processing steps are documented in Sec. 3.1 for futures data and in Sec. 3.6 for options data. The pipeline is reproducible, given access to the same sources.

4 Experiments

TODO: decide whether the per-split distributional statistics belong here or in Ch. 4.
Numbers already computed: Split Std Kurtosis Skew Train 0.0377 19.11 0.71 Validation
0.0515 2.58 0.59 Test 0.0402 13.42 0.92 Point: validation is a different regime from test
(2.58 vs 13.42), which explains why validation Sharpe does not predict test Sharpe on
TTF.

5 Discussion

TODO: Limitations: a two-return construction was possible from opening and settlement prices but not pursued — returns would need aggregating back to daily for the Monte Carlo, and would mix a traded price with an exchange mark.

TODO: Since each condition appears once in the data with a single realized target, the empirical conditional distribution is a point mass. Vuletić et al. (2024) note that forecasts close to the target therefore receive high scores from the discriminator, which encourages mode collapse in a classical GAN setup

6 Conclusion and Outlook

References

- Arjovsky, Martin, Soumith Chintala, and Léon Bottou (Dec. 6, 2017). *Wasserstein GAN*. DOI: 10.48550/arXiv.1701.07875. arXiv: 1701.07875 [stat.ML]. URL: <http://arxiv.org/abs/1701.07875> (visited on 08/15/2026). Pre-published.
- Barsha, Farhat Lamia and William Eberle (June 2025). “An In-Depth Review and Analysis of Mode Collapse in Generative Adversarial Networks”. In: *Machine Learning* 114.6, p. 141. ISSN: 0885-6125, 1573-0565. DOI: 10.1007/s10994-025-06772-7. URL: <https://link.springer.com/10.1007/s10994-025-06772-7> (visited on 08/14/2026).
- Brown, Stephen P.A. and Mine K. Yücel (Apr. 2008). “What Drives Natural Gas Prices?” In: *The Energy Journal* 29.2, pp. 45–60. ISSN: 0195-6574, 1944-9089. DOI: 10.5547/ISSN0195-6574-EJ-Vol29-No2-3. URL: <https://journals.sagepub.com/doi/10.5547/ISSN0195-6574-EJ-Vol29-No2-3> (visited on 08/12/2026).
- Chung, Junyoung et al. (Dec. 11, 2014). *Empirical Evaluation of Gated Recurrent Neural Networks on Sequence Modeling*. DOI: 10.48550/arXiv.1412.3555. arXiv: 1412.3555 [cs.NE]. URL: <http://arxiv.org/abs/1412.3555> (visited on 08/16/2026). Pre-published.
- European Commission (Nov. 22, 2022). *Questions and Answers on the Gas Market Correction Mechanism*. QANDA/22/7066. Brussels: European Commission. URL: https://ec.europa.eu/commission/presscorner/detail/en/qanda_22_7066.
- Goodfellow, Ian (Apr. 3, 2017). *NIPS 2016 Tutorial: Generative Adversarial Networks*. DOI: 10.48550/arXiv.1701.00160. arXiv: 1701.00160 [cs.LG]. URL: <http://arxiv.org/abs/1701.00160> (visited on 08/14/2026). Pre-published.
- Goodfellow, Ian et al. (Oct. 22, 2020). “Generative Adversarial Networks”. In: *Communications of the ACM* 63.11, pp. 139–144. ISSN: 0001-0782, 1557-7317. DOI: 10.1145/3422622. URL: <https://dl.acm.org/doi/10.1145/3422622> (visited on 08/13/2026).
- Goodfellow, Ian J. et al. (June 10, 2014). *Generative Adversarial Networks*. DOI: 10.48550/arXiv.1406.2661. arXiv: 1406.2661 [stat.ML]. URL: <http://arxiv.org/abs/1406.2661> (visited on 08/13/2026). Pre-published.
- Heather, Patrick (Mar. 2022). *A Series of Unfortunate Events Explaining European Gas Prices in 2021 The Role of the Traded Gas Hubs*. OIES Energy Insight 111. Oxford: Oxford Institute for Energy Studies. URL: <https://www.oxfordenergy.org/wpcms/wp-content/uploads/2022/03/Insight-111-Explaining-European-gas-prices-in-2021-the-role-of-the-traded-gas-hubs.pdf>.
- (2026). *European Traded Gas Hubs: Resilience in the Face of Adversity*. OIES Paper NG 206. Oxford: The Oxford Institute for Energy Studies. 1 p. ISBN: 978-1-78467-299-7.
- Hochreiter, Sepp and Jürgen Schmidhuber (Nov. 1, 1997). “Long Short-Term Memory”. In: *Neural Computation* 9.8, pp. 1735–1780. ISSN: 0899-7667, 1530-888X. DOI: 10.1162/neco.1997.9.8.1735. URL: <https://direct.mit.edu/neco/article/9/8/1735-1780/6109> (visited on 08/16/2026).

References

- Koochali, Alireza et al. (2019). “Probabilistic Forecasting of Sensory Data With Generative Adversarial Networks – ForGAN”. In: *IEEE Access* 7, pp. 63868–63880. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2915544. URL: <https://ieeexplore.ieee.org/document/8717640/> (visited on 08/15/2026).
- Mirza, Mehdi and Simon Osindero (Nov. 6, 2014). *Conditional Generative Adversarial Nets*. DOI: 10.48550/arXiv.1411.1784. arXiv: 1411.1784 [cs.LG]. URL: <http://arxiv.org/abs/1411.1784> (visited on 08/14/2026). Pre-published.
- Obadi, Saleh Mothana and Matej Korcek (Oct. 10, 2020). “DRIVING FUNDAMENTALS OF NATURAL GAS PRICE IN EUROPE”. In: *International Journal of Energy Economics and Policy* 10.6, pp. 318–324. ISSN: 21464553. DOI: 10.32479/ijeep.10192. URL: <https://econjournals.com/index.php/ijeep/article/view/10192> (visited on 08/12/2026).
- Vuletić, Milena, Felix Prenzel, and Mihai Cucuringu (Feb. 2024). “Fin-GAN: Forecasting and Classifying Financial Time Series via Generative Adversarial Networks”. In: *Quantitative Finance* 24.2, pp. 175–199. ISSN: 1469-7688, 1469-7696. DOI: 10.1080/14697688.2023.2299466. URL: <https://www.tandfonline.com/doi/full/10.1080/14697688.2023.2299466> (visited on 08/11/2026).

Documentation of AI Tool Usage

Declaration of Authorship

Hereby, I declare that I have independently prepared the examination performance or thesis in the above-mentioned module – for group work, the portion of the work marked with my contribution – and have not used any sources or aids other than those indicated. Texts, ideas, concepts, graphics, etc., taken directly or indirectly from external sources in my statements have been clearly identified as such and provided with full references to the respective source.

All quotations, as well as all sections written or modified using AI-based tools, have been checked for correctness and marked accordingly. Documentation of the AI tools used, along with their purpose of use, is attached to the work. I take responsibility for the quality and all contents of the present work, and my creative influence predominates.

Furthermore, I assure that I have exclusively used AI-supported tools that the examiner explicitly identified as approved aids. The work has not been submitted in the same or a similar form in any other examination.

Fulda, August 19, 2026

Yeva Galstyan